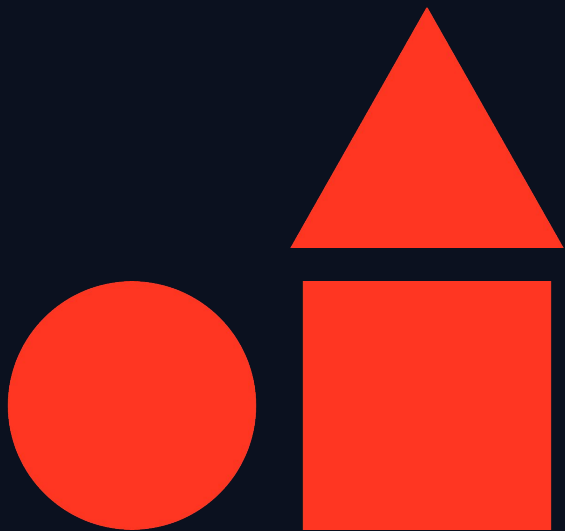




Omnigent

A Meta-Harness for AI Agents



20,000+

global customers

\$4B+

in revenue

Free Edition
now available

Inventor of the **lakehouse**

&

Pioneer of **generative AI**



databricks
Data & AI Platform

Gartner-recognized Leader

Database Management Systems
Data Science & Machine Learning
Platforms

Creator of OSS



DELTA LAKE

mlflow

APACHE
Spark



Unity Catalog

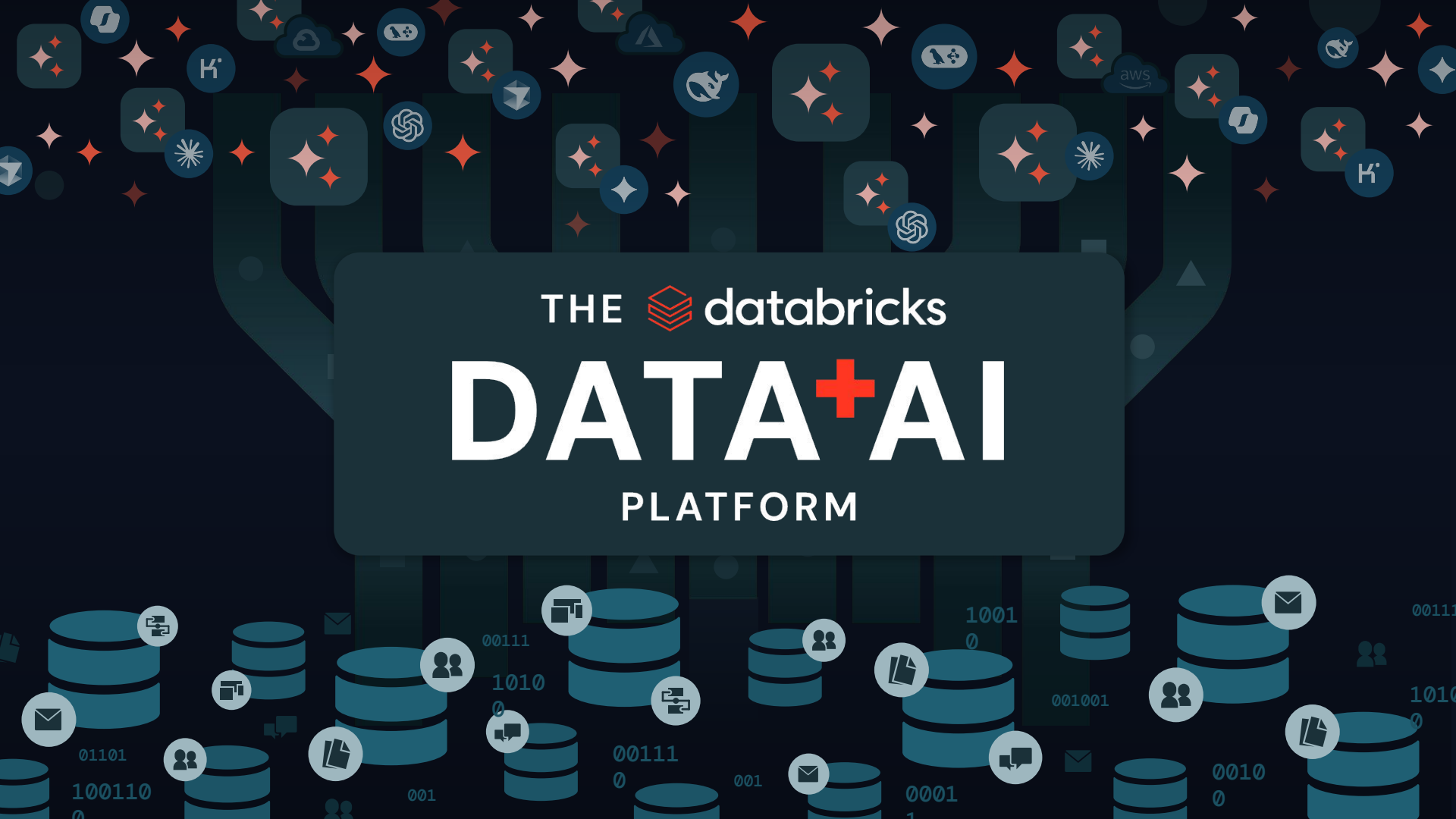


DSPy



Omnigent





THE  databricks

DATA+AI

PLATFORM



THE **DATA+AI** PLATFORM

Agentic Apps



Apps



Lakewatch



CustomerLake

Agentic Work



Genie



ONE



AGENTS



CODE



APP BUILDER



ZERO OPS



Agent Bricks



Omnigent

Agentic Development



Genie Ontology

Unified Governance



Unity Catalog



Unity AI Gateway

Agentic Data



Lakeflow



Lakehouse



Lakebase

Open Infrastructure

Open Format Data Lake



Any Cloud



Any Model



Any Data





THE **DATA+AI** PLATFORM

Agentic Apps



Apps



Lakewatch



CustomerLake

Agentic Work



Genie

ONE

AGENTS

CODE

APP BUILDER

ZERO OPS



Agent Bricks



Omnigent

Agentic Development



Genie Ontology

Unified Governance



Unity Catalog



Unity AI Gateway

Agentic Data



Lakeflow



Lakehouse



Lakebase

Open Infrastructure

Open Format Data Lake

DELTA LAKE

ICEBERG

Any Cloud

Any Model

Any Data

Agents are rapidly evolving

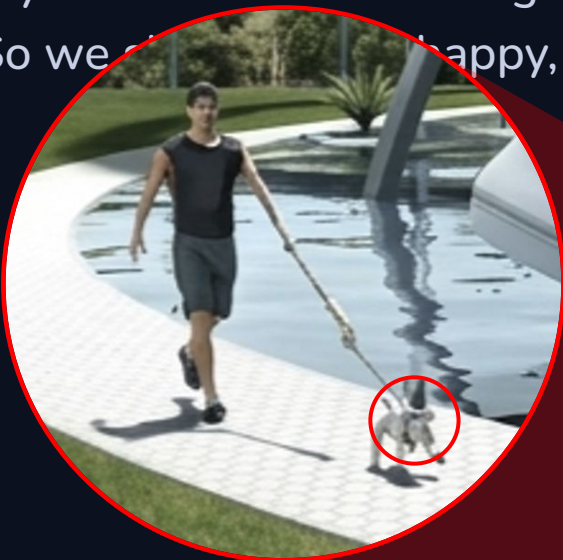


Agents are really starting to work

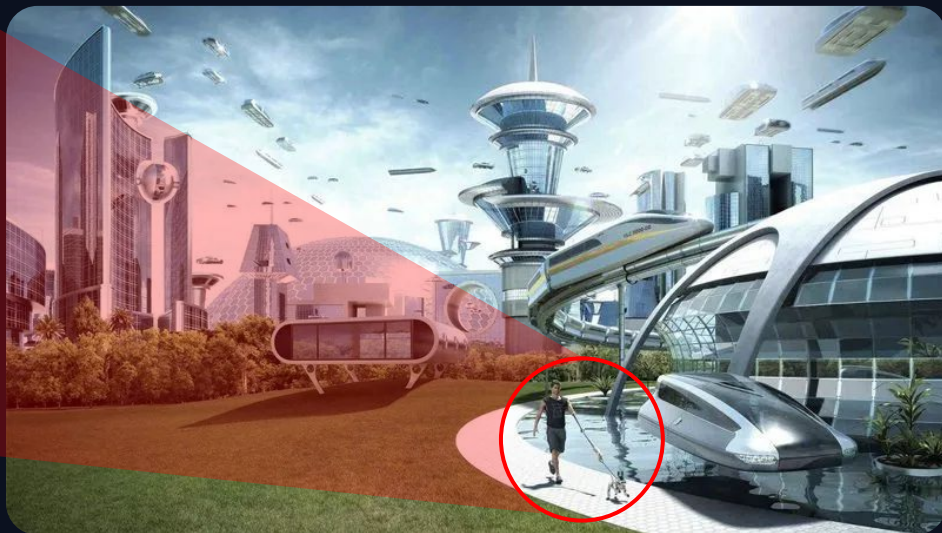
- Coding agents are the biggest success, but same in other fields
- If you don't like current agents, vibe-code your own
- So we should all be happy, right?

Agents are really starting to work

- Coding agents are the biggest success, but same in other fields
- If you don't like current agents, vibe-code your own
- So we're all happy, right?



But there's a problem!



Society if agents really worked

Problem: the layer around agents (the harness)

Everyone can build their own agent harness, and everyone has

CLAUDE
CODE



Codex

 Google Antigravity

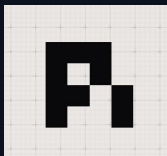
 CURSOR

 goose

 replit

opencode

 Windsurf



 KIRO

 TRAE

 Kilo Code

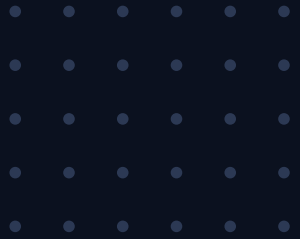
The layer around agents (the harness)

Everyone can build their own agent harness, and everyone has

This makes interoperability hard, and causes three issues:

- How do I **compose** agents with different harnesses to do a task?
- How do I **collaborate** across humans and agents?
- How do I **control** all my agents for cost & security?

We saw these problems repeatedly within Databricks!



We need a new, open layer
above the harness level.

A meta-harness





Omnigent

A Meta-Harness for AI Agents



Omnigent

Omnigent

New session

Search

My sessionsShared with me

Sessions

Deploy omnigent server on apps

ponytail Can we display a counter of th...
I* fix/file-line-diff-counter3m

Fork of Can you help me check on how ag...
I* feat/unblock-async-ops4m

Can you help make 4 short bullet point...16m

Write me an example of a custom agent...57m

https: omnigent.ai docs omnibox Can y...1h

Hey is it possible to implement a policy ...1h

Hey for some reason even though I hav...
I* fix/pl-agent-cannot-start1h

Help me check on how for managed o...2h

Can you help me check on how agent s...2h

Implemnt a feature for me to clear a c...
I* feat/clear-chat-within-session10h

Can you write me an example on how I ...21h

can you update the branch fix-historica...22h

Hey I noticed that the working... animat...
I* fix/persist-working-animation1d

Can you check the codebase on how c...
I* fix/remember-codex-mode-reconnect-host1d

Settings

Fork of Can you help me check on how ag...

3 2 seconds: Sync Request B starts
4 3 seconds: The next message for Request A becomes available
5 12 seconds: Sync Request B finishes
6 12 seconds: Request A sends its next message

From the user's point of view, the stream appears frozen for nine seconds even though its next message was already available.

Multiple Server Workers

Running multiple server workers can reduce the impact because another process may handle the unrelated request.

However, this does not fully solve the problem:

- Requests assigned to the blocked worker are still delayed.
- Each additional worker uses more memory.
- Each worker may load its own copy of the agent.
- Users must increase the number of workers to avoid delays caused by one synchronous request.

The server should remain responsive within each worker rather than relying only on additional workers.

Expected Behavior

A synchronous agent function should still wait for its own work to finish. However, that waiting should happen separately so the server can continue asynchronous work.

The desired flow is:

text

1 0 seconds: Async Request A starts and waits
2 1 second: Sync Request B starts and delays
3 2 seconds: Request A becomes ready and continues

Ask the agent anything...

GG230D22D3 I* feat/unblock-async-ops

Codex 97%

ChatTerminal

FilesAgentsShellsBrowserserver.py

mlflow/genai/agent_server/server.py

```
10 hidden lines
11 import uvicorn
12 from fastapi import FastAPI, HTTPException, Request
13 from fastapi.responses import Response, StreamingResponse
14 from starlette.concurrency import iterate_in_thread
15
16 import mlflow
17 from mlflow.genai.agent_server.utils import get_req
302 hidden lines
319 320 if inspect.iscoroutinefunction(func):
321     result = await func(request)
322 else:
323     result = func(request)
324     result = await run_in_threadpool(func, request)
325
326 result = self.validator.validate_and_stream(result)
327 if self.agent_type == "ResponsesAgent":
328     all_chunks.append(chunk)
329     yield f"data: {json.dumps(chunk)}\n\n"
330 else:
331     for chunk in func(request):
332         stream = await run_in_threadpool(func, request)
333         async for chunk in iterate_in_threadpool(stream):
334             chunk = self.validator.validate_and_stream(chunk)
335             all_chunks.append(chunk)
336             yield f"data: {json.dumps(chunk)}\n\n"
337 70 hidden lines
```

4:02

127.0.0.1:52138

What should we do?

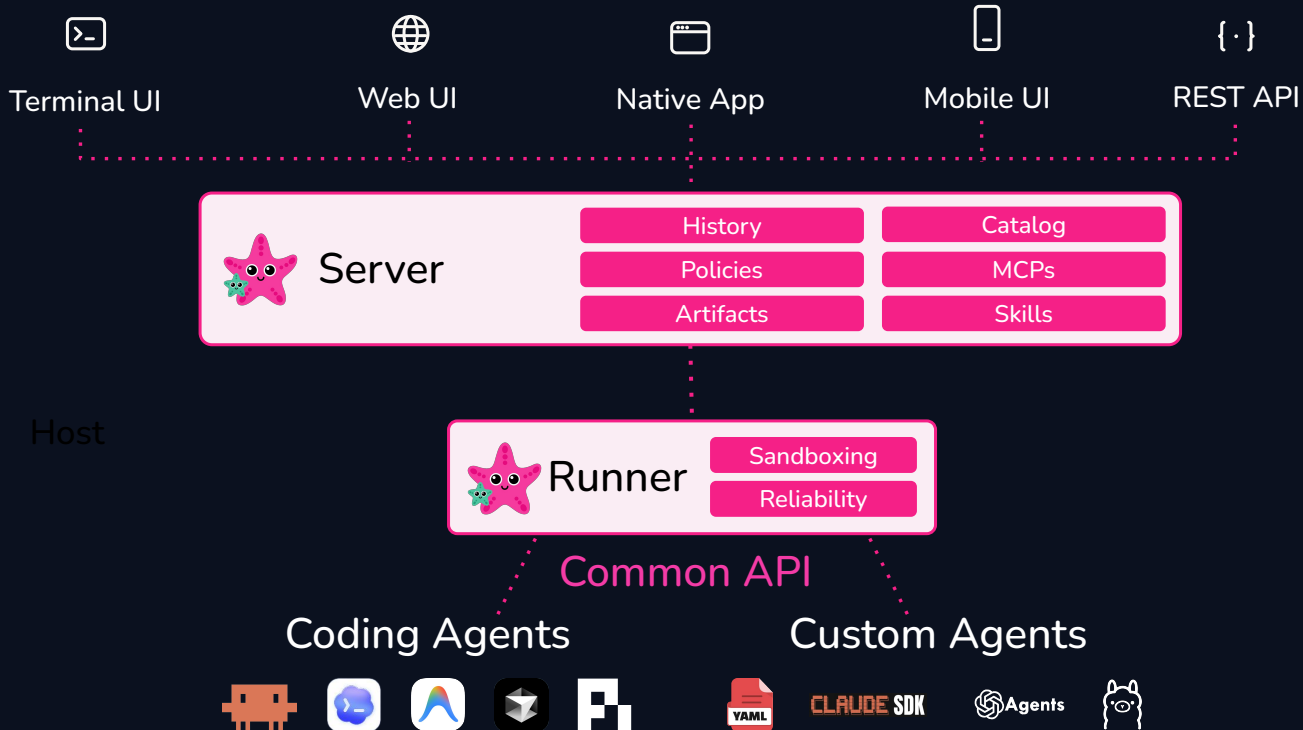
Describe a task to start a new session...

 Claude Code





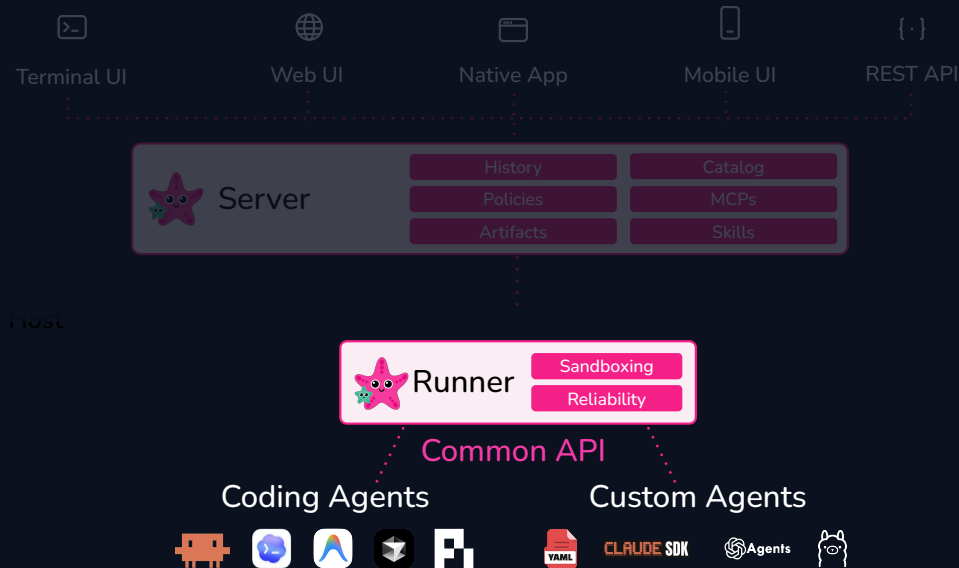
Omnigent



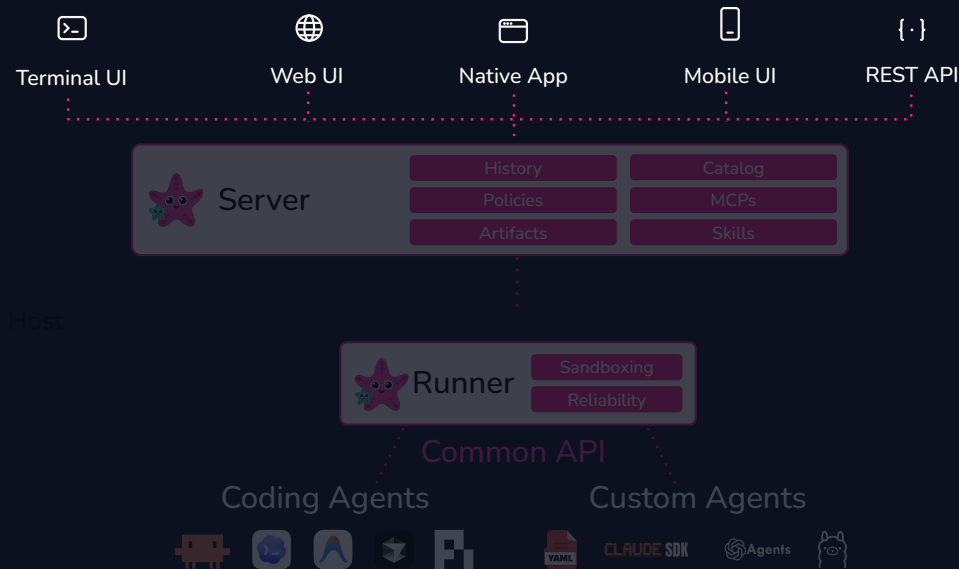
What Omnigent provides

Composition

- Combine agents using multiple harnesses / LLMs
- Switch agent harness



What Omnigent provides



Composition

- Combine agents using multiple harnesses / LLMs
- Switch agent harness

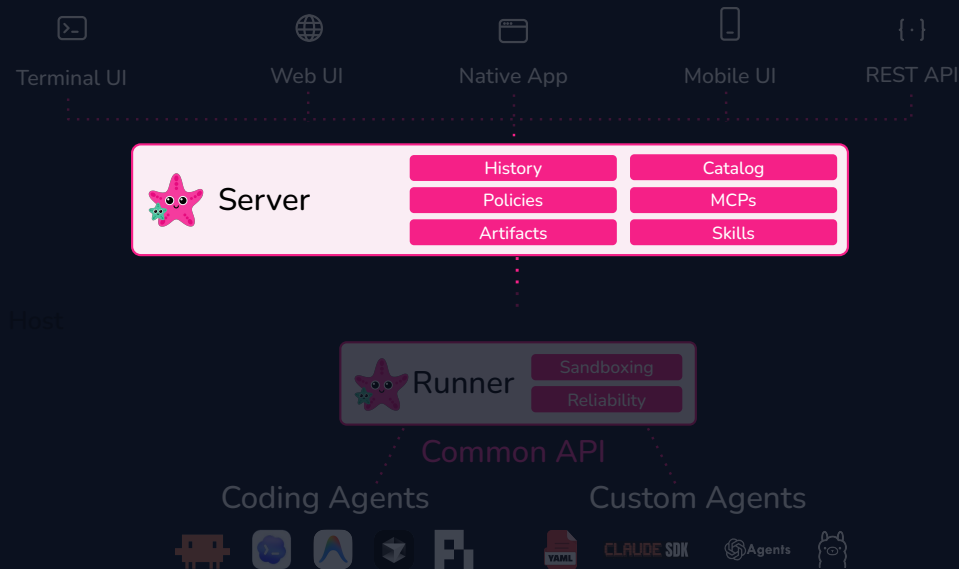


Collaboration

- Share and co-drive live sessions with other people
- Comment on files / docs



What Omnigent provides



Composition

- Combine agents using multiple harnesses / LLMs
- Switch agent harness



Collaboration

- Share and co-drive live sessions with other people
- Comment on files / docs



Control

- Powerful **contextual** policies for security and cost
- Strong OS sandbox





Omnigent Composition

Use 13+ harnesses out-of-the-box

No migration — Omnigent leverages your setup with coding agents on your machine.

Get the best of each — Use the best tools and capabilities from each harness through collaboration between agents

Extensible framework — Harness Plugin SDK allows for harness integrations without changes to Omnigent.

Configure harnesses

> Claude	✓ Databricks (oss)
Codex	✓ Databricks (oss)
Cursor	× Not installed
OpenCode	× Not installed
Hermes	× Not installed
Pi	✓ Databricks (oss)
Antigravity	× Not installed
Qwen Code	× Not installed
Goose	× Not installed
Copilot	× Not installed
Kiro	× Not installed
Kimi Code	× Not installed
Custom ACP agent	None configured
Quit	





Omnigent Custom Agents

Define, orchestrate, and govern multi-agent teams in a single config

Orchestrate multiple agents — Combine roles, models, tools, and harnesses in one team.

Portable configuration — Define and reuse complete multi-agent workflows with YAML.

Flexible and governed — Deploy per session or as shared server templates with consistent policies and sandboxing.

```
1 spec_version: 1
2
3 name: research-team
4 description: A supervisor coordinating specialized agents.
5
6 prompt: |
7   Delegate research, analysis, and review.
8   Combine their results into one concise answer.
9
10 executor:
11   harness: claude-sdk
12   model: your-model
13
14 tools:
15   researcher:
16     type: agent
17     description: Finds relevant information.
18     prompt: Research the topic and summarize key facts.
19     executor:
20       harness: claude-sdk
21       pass_history: true
22
23   analyst:
24     type: agent
25     description: Identifies insights and patterns.
26     prompt: Analyze the research and produce recommendations.
27     executor:
28       harness: codex
29       pass_history: true
30
31   reviewer:
32     type: agent
33     description: Checks quality and accuracy.
34     prompt: Review the work and flag unsupported conclusions.
35     executor:
36       harness: openai-agents
37       pass_history: true
38
39 async: true
40 cancellable: true
```





Omnigent Contextual Policy

Policies that sees the
whole agent execution.

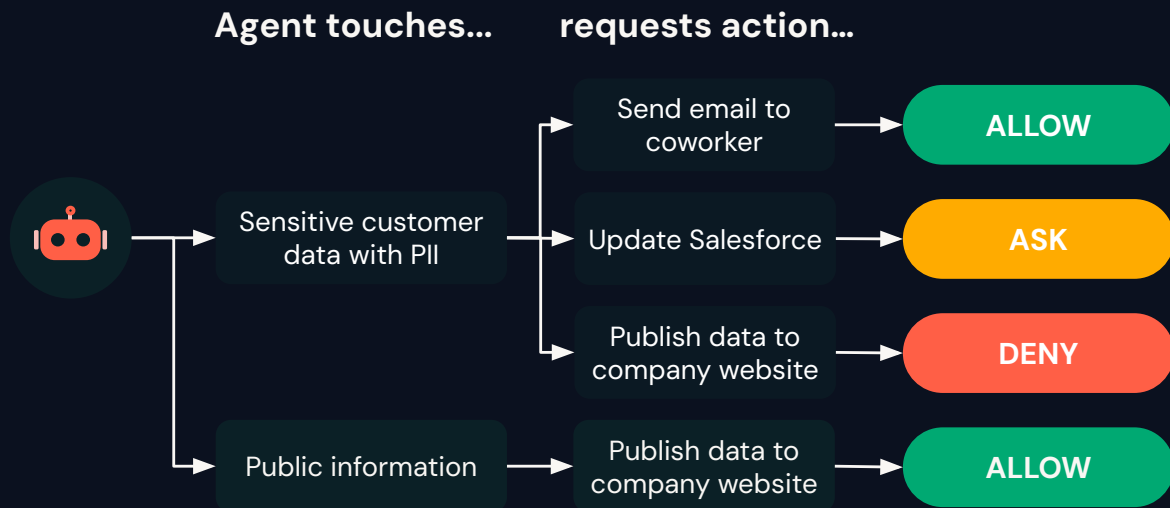
Context-aware decisions —

Evaluates both the data involved and the requested action.

Granular enforcement — Allows, denies, or requests approval for each action.

Sensitive data protection —

Prevents high-risk actions, and allow safe actions to proceed automatically.



Complementary governance for your agent harness



Omnigent

Governs the agent execution and trajectory

Governs the full run, not isolated calls. Stateful policies span the entire multi-step session — cumulative spend, tool-call counts & a running risk score carried across steps.

Governs local actions with no network egress. A gateway cannot inspect local operations (e.g. `ls`, `rm -rf`, `cat`). Omnigent gates them in-loop, before they run.

Leverage OS sandbox to control your harness's local files and network access



Unity AI Gateway

Governs the individual requests to model and tool

Enforces governance at the endpoint level.

Intercepts every request / response to models & MCP services

Permissions & Audit through Unity Catalog.

Control access to AI assets with Unity Catalog. Log requests & responses to Delta tables for monitoring.

Aggregated spend caps & cost attribution.

Per-user thresholds and hard limits. Cost broken down by service, users, principals, tag





Omnigent Collaboration

Bring in others to *collaborate* and *comment* on your session!

Share this session

Invite others to view or collaborate on this session.

Please be careful when sharing. Granting edit access will allow others to interact with your agent. Read access will allow for reading of all session outputs.

Workspace access

Any workspace user can view this session

NAME		PERMISSION	
simonj	@databricks.com	Read	
anth	@databricks.com	Owner	
rl	@databricks.com	Edit	

User ID

Level

stewart

Read

Grant

Stewart

stewart

@databricks.com

Copy link

Done

Files 14 Agents 1 Shells Browser AppShell.tsx

< 5/14 > web/src/shell/AppShell.tsx

197 hidden lines

```
567 574 // the page it drives). The screenshot ar
568 575 // event, so we stash the latest per conve
569 576 const designShotRef = useRef<string, s
570 - const boundAgentId = boundAgent?.id ?? nu
571+ const boundAgentId = boundAgent?.id ?? ac
572+ const handleClearChat = useCallback(() =>
573+   if (!conversationId || !boundAgentId ||
574+     const clear = isNativeWrapper
575+       ? useChatStore.getState().send("/clear
576+       : clearSessionChat(conversationId).the
577+       navigate('/c/${replacement.id}',
578+     ));
579+   void clear.catch((err: unknown) => {
580+     showToast(err instanceof Error ? err.m
581+   ));
582+ }, [boundAgentId, clearDisabled, conversa
583+
584+ useEffect(() => {
585+   if (!supportsBrowser()) return;
586+   const w = window as unknown as f
```

Comments

Address All

Open 1 Addressed

const handleClearChat = useCallback(()=> { if (!...
Could handleClearChat delegate to a shared clear-chat
action? The native/API branching, agent resolution,
navigation, and error handling are domain workflow rather
than shell concerns, and the /clear composer path ...
Show more
an @databricks.com
0:18 Today Edit Delete





Omnigent Collaboration

Bring in others to *collaborate* and *comment* on your session!

Session sharing

Control whether users on this server can share sessions with others. Applies server-wide and takes effect immediately. Changes affect only new shares — existing grants (including already-public sessions) keep working until revoked.



On

Anyone with manage access can share a session at any level (read, edit, or manage) and toggle public / workspace read.



Read only

New shares are capped at read (view) access. Edit and manage grants are rejected.



Read only (restricted)

Read-only, and sessions whose working directory is a home directory or the filesystem root cannot be shared at all — not even read.



Off

Sharing is disabled. No new grants can be created and the Share control is hidden.

Public access

Allow sharing a session with anyone who has the link (public read access). When off, the Share dialog's "Public access" toggle is hidden and new public grants are rejected; sessions already shared publicly stay public until revoked.



Session Sharing option can be configured from the server side by the *admins*





Omnigent on databricks

Open Source

Deployment of server and host handled separately

- Deployable on Databricks Apps
- Leverage Databricks Identities, and Lakebase for session states.

Need separate governance for LLMs, MCPs, and Skills

- Governance can be done through Unity AI Gateway

Sandbox instance for isolated execution need to be sourced separately (e.g. Modal, Daytona)

Managed On Databricks

A **Databricks-managed Omnigent server** that integrates with your workspace identity provider.

- *Managed server can interact with local host, or host running on other sandbox / deployment services*

LLMs, MCPs, Skills access **governed by Unity AI Gateway**

Databricks Sandbox as Omnigent host, providing secure, collaborative agent coding and knowledge work

Same core codebase as Open Source





Omnigent Auth & SSO

Open Source, or Deploy on Apps

Mode	When to use
<u>Built-in accounts</u>	Standalone deploy, no external IdP. Username/password with invite links.
<u>Single sign-on (OIDC)</u>	Your own IdP: Google, GitHub, Okta, Microsoft.
<u>Header-based auth</u>	Behind an existing SSO proxy that injects <code>X-Forwarded-Email</code> .



User management, SSO, OIDC authentication **integrates with your workspace identity provider automatically**





Omnigent Collaboration

Bring in others to *collaborate* and *comment* on your session!

Share this session [X]

Invite others to view or collaborate on this session.

Please be careful when sharing. Granting edit access will allow others to interact with your agent. Read access will allow for reading of all session outputs.

Workspace access [Toggle]

Any workspace user can view this session

NAME	PERMISSION	
nston@databricks.com	Read	
y.ivan@databricks.com	Owner	
garris@databricks.com	Edit	

User ID Level

str rpa Read [Grant]

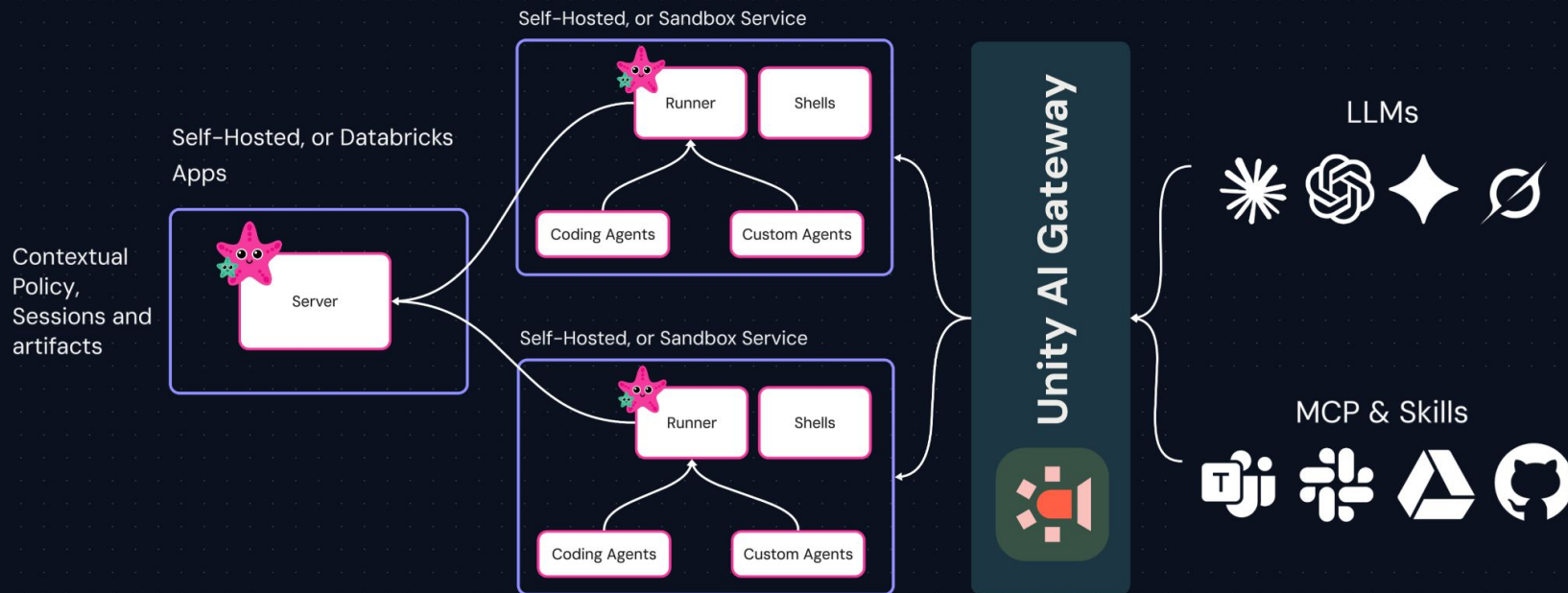
St erpa s sherpa@databricks.com

[Copy link] [Done]

1. **Managed Omnigent Server** has built-in control to only allow access provisioning to users of the workspace
2. Read access allows other users to comment on your session



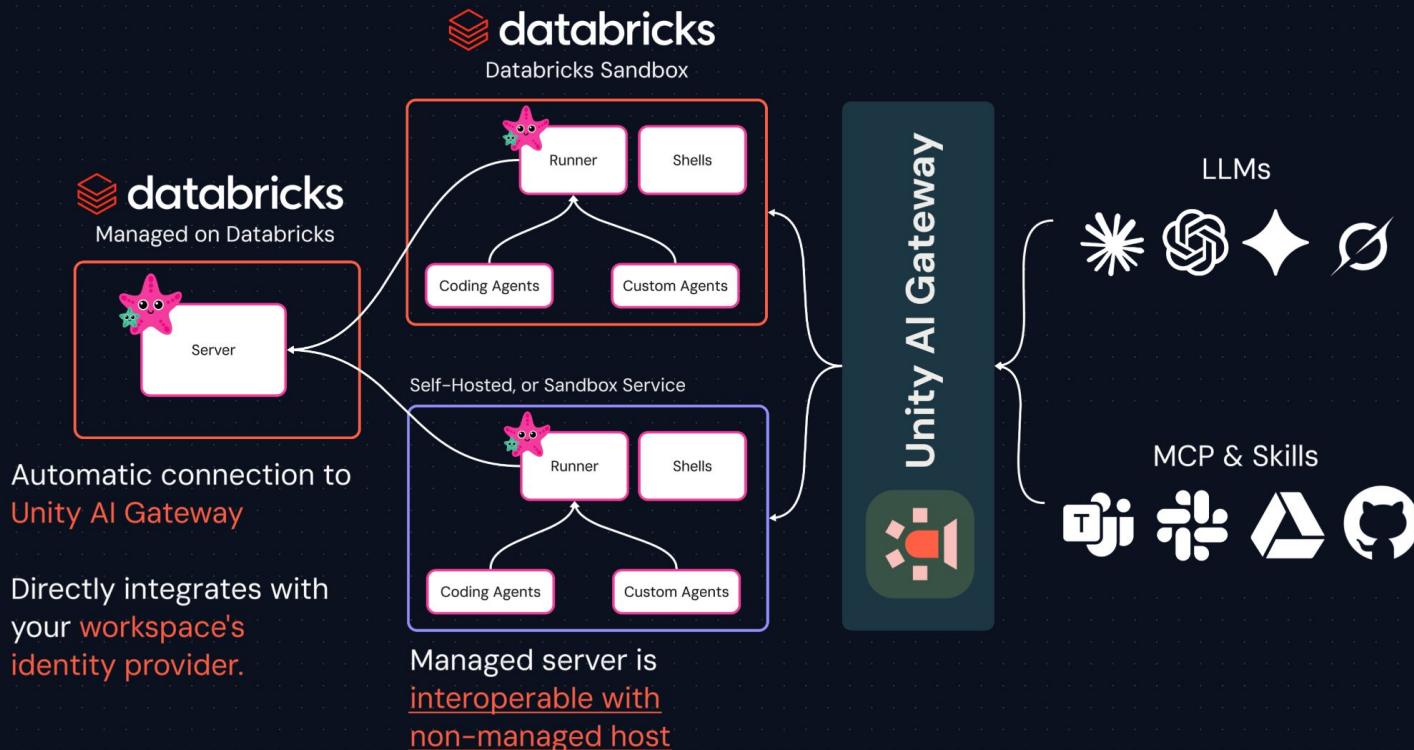
Govern access to LLMs, MCPs, Skills for Omnigent OSS with Unity AI Gateway



Access to LLMs, MCPs, Skills
Governed by Unity Catalog

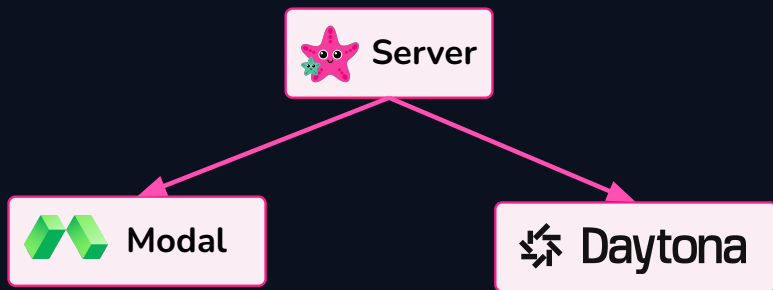


Managed Omnigent integrates directly with your databricks environment, and interoperates with OSS





Cloud & OS Sandbox



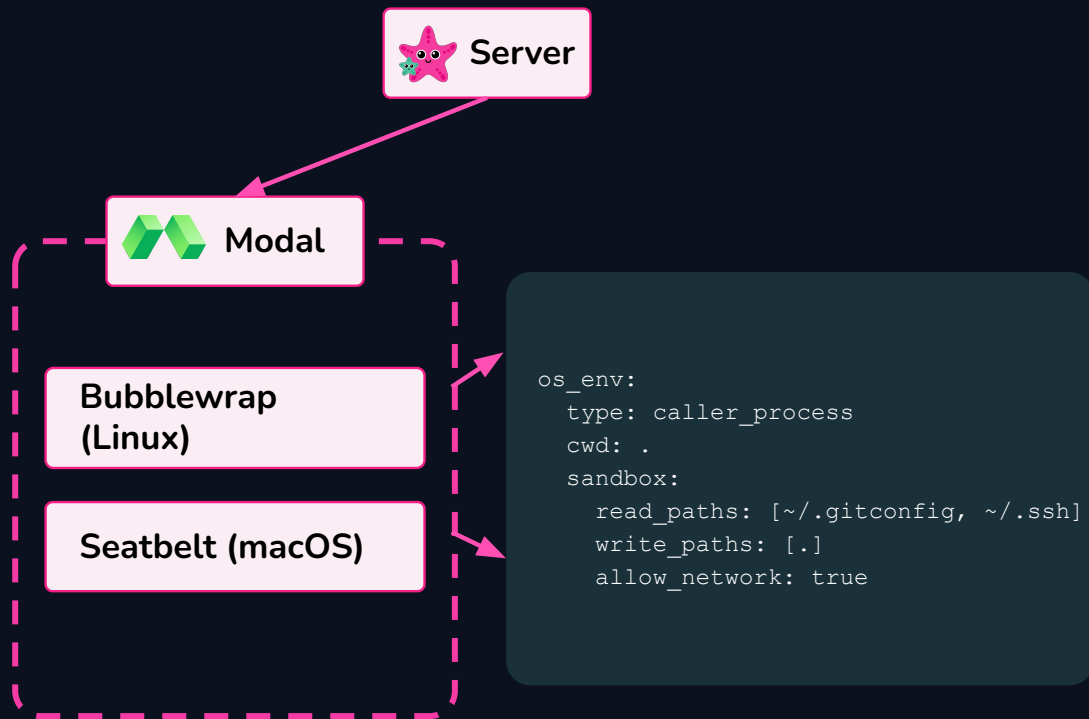
Cloud sandbox connectivity

- Always-on Availability
- Scalability
- Reproducible Environments





Cloud & OS Sandbox



Cloud sandbox connectivity

- Always-on Availability
- Scalability
- Reproducible Environments

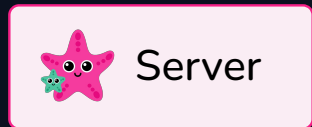
OS sandbox with **Omnibox**

- Filesystem isolation
- Network controls
- Credential protection
- Process containment

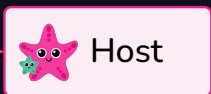




Omnibox: OS Sandbox



*Server can enforce OS
Sandbox configs to all host*



*Host can
define its
own OS
Sandbox
configs*

```
# Inspect the sandbox
ls -la

# Succeeds, but /tmp is private to this
sandbox
echo "temporary" > /tmp/omnibox-test

# Should fail or be invisible
ls ~/.ssh
cat ~/.aws/credentials
echo "escape" > ../outside-project.txt

# fail when direct networking is disabled
curl https://example.com
```

Configurable OS sandbox,
enforced at the server and host
layer

Filesystem isolation — Limits agents to
approved files and writable paths.

Network controls — Blocks direct access
or restricts traffic to approved destinations.

Credential protection — Prevents agents
from directly reading sensitive host
credentials.

Process containment — Ensures agent
subprocesses inherit the same security
boundaries.





Where we're headed

1. **Reliability, stability, & performance:** we want to make Omnigent amazing to use day in and out.
 - We're rolling out Omnigent for all of Databricks engineering by the end of August!
2. **Extend where you can use it:** Windows support, iOS & Android apps, Slack integration
3. **Community:** sandbox plugin SDK, review & merge PRs from OSS members
4. **Better custom agent & policy management:** share custom agents & policies more easily, enforce usage on the server w/ admin controls

